Lecture 7. Strings

stringr basics, regular expressions, tools for matching

Contents

7.1 String basics **2**
- 7.1.1 Length, combining, subsetting . . . 2

7.2 Regular expressions **3**
- 7.2.1 Basic matches . . . 3
- 7.2.2 Anchors and character classes . . . 3
- 7.2.3 Repetition . . . 3
- 7.2.4 Grouping and backreferences . . . 4

7.3 Tools **4**
- 7.3.1 Detecting matches . . . 4
- 7.3.2 Extracting matches . . . 5
- 7.3.3 Grouped matches . . . 5
- 7.3.4 Replacing and splitting . . . 5

7.4 Selected exercises **6**

7.1 String basics

Motivation. Information packed into text is pulled out with stringr and regular expressions.

`stringr` (core tidyverse): consistent `str_*` functions.

Double or single quotes (single when the string contains double quotes):

```
string1 <- "This is a string"
string2 <- 'If I want to include a "quote" inside a string, I use single quotes'
```

A missing closing quote gives the `+` prompt; press Esc.

Escapes. `"\""` is a double quote, `"\\"` a backslash. Printing shows escapes; `writeLines()` the raw contents:

```
x <- c("\"", "\\")
x
```

```
## [1] "\"" "\\"
```

```
writeLines(x)
```

```
## "
## \
```

Also `"\n"` (newline), `"\t"` (tab); see `?"'"`. Unicode: `"\u"` + code, e.g. `"\uc804\uc0b0\ud1b5\uacc4"` (course name in Hangul). Character vector: `c("one", "two", "three")`.

7.1.1 Length, combining, subsetting

```
str_length(c("a", "R for data science", NA))
```

```
## [1]  1 18 NA
```

```
str_c("x", "y", sep = ", ")
```

```
## [1] "x, y"
```

```
str_c("prefix-", c("a", "b", "c"), "-suffix")     # vectorised, recycles
```

```
## [1] "prefix-a-suffix" "prefix-b-suffix" "prefix-c-suffix"
```

```
str_c(c("x", "y", "z"), collapse = ", ")          # collapse a vector into one string
```

```
## [1] "x, y, z"
```

`str_c()` drops length-0 objects: in `str_c("Good ", time_of_day, " ", name, if (birthday) " and HAPPY BIRTHDAY", ".")` the clause vanishes when `birthday` is `FALSE`. `NA` is contagious; `str_replace_na()` prints `"NA"`.

`str_sub(x, start, end)`: inclusive positions, negative from the end, out of range allowed:

```
x <- c("Apple", "Banana", "Pear")
str_sub(x, 1, 3)
```

```
## [1] "App" "Ban" "Pea"
```

```
str_sub(x, -3, -1)
```

```
## [1] "ple" "ana" "ear"
```

```
str_sub("a", 1, 5)
```

```
## [1] "a"
```

```
str_sub(x, 1, 1) <- str_to_lower(str_sub(x, 1, 1))   # assignment form
x
```

```
## [1] "apple"  "banana" "pear"
```

Positions count characters, not bytes (fine for Hangul).

7.2 Regular expressions

Motivation. A terse language for patterns such as "any digit" or "a repeated letter".

`str_view()` marks matches with `< >` (`str_view_all()` is deprecated).

7.2.1 Basic matches

Exact strings; `.` is any character but newline:

```
x <- c("apple", "banana", "pear")
str_view(x, "an")
```

```
## [2] | b<an><an>a
```

```
str_view(x, ".a.")
```

```
## [2] | <ban>ana
## [3] | p<ear>
```

A literal `.` is the regex `\.`, written as the string `"\\."`; a literal backslash is the regex `\\`, the string `"\\\\"`:

```
str_view(c("abc", "a.c", "bef"), "a\\.c")
```

```
## [2] | <a.c>
```

```
writeLines("a\\b")
```

```
## a\b
```

```
str_view("a\\b", "\\\\")
```

```
## [1] | a<\>b
```

7.2.2 Anchors and character classes

`^` start, `$` end ("begin with power, end up with money"); both force a complete match:

```
x <- c("apple pie", "apple", "apple cake", "pine apple")
str_view(x, "^apple$")
```

```
## [2] | <apple>
```

`\d` digit, `\s` whitespace, `[abc]` one of a, b, c, `[^abc]` none of them; in strings `"\\d"` etc. Alternation `|`, delimited by parentheses:

```
str_view(c("grey", "gray"), "gr(e|a)y")
```

```
## [1] | <grey>
## [2] | <gray>
```

7.2.3 Repetition

`?` means 0 or 1, `+` 1 or more, `*` 0 or more; `{n}` exactly n, `{n,}` n or more, `{,m}` at most m, `{n,m}` between n and m. **Greedy** (longest match) by default; a trailing `?` makes it **lazy**.

```
x <- "1888 is the longest year in Roman numerals: MDCCCLXXXVIII"
str_view(x, "CC+")
```

```
## [1] | 1888 is the longest year in Roman numerals: MD<CCC>LXXXVIII
```

```
str_view(x, "C[LX]+")
```

```
## [1] | 1888 is the longest year in Roman numerals: MDCC<CLXXX>VIII
```

```
str_view(x, "C{2,3}")
```

```
## [1] | 1888 is the longest year in Roman numerals: MD<CCC>LXXXVIII
```

```
str_view(x, "C{2,3}?")
```

```
## [1] | 1888 is the longest year in Roman numerals: MD<CC>CLXXXVIII
```

7.2.4 Grouping and backreferences

Parentheses define **groups**, referred to as `\1`, `\2`, ... Fruits with a repeated pair:

```
str_subset(fruit, "(..)\\1")
```

```
## [1] "banana"      "coconut"     "cucumber"    "jujube"      "papaya"      "salal berry"
```

7.3 Tools

Motivation. Detect, extract, replace, split: one function each.

7.3.1 Detecting matches

`str_detect()` is logical, so `sum()`, `mean()` count; `words`: 980 common words.

```
str_detect(c("apple", "banana", "pear"), "e")
```

```
## [1]  TRUE FALSE  TRUE
```

```
sum(str_detect(words, "^t"))          # how many words start with t?
```

```
## [1] 65
```

```
mean(str_detect(words, "[aeiou]$"))   # proportion ending with a vowel
```

```
## [1] 0.2765306
```

```
str_subset(words, "x$")               # same as words[str_detect(words, "x$")]
```

```
## [1] "box" "sex" "six" "tax"
```

In a data frame: `filter(str_detect(word, "x$"))`. `str_count()` counts matches:

```
tibble(word = words) %>%
  mutate(vowels = str_count(word, "[aeiou]"),
         consonants = str_count(word, "[^aeiou]"))
```

```
## # A tibble: 980 x 3
##   word     vowels consonants
##   <chr>     <int>      <int>
## 1 a             1          0
## 2 able          2          2
## 3 about         3          2
## 4 absolute      4          4
## 5 accept        2          4
## # i 975 more rows
```

Several `str_detect()` calls with logical operators beat one complex regex: no vowels is `!str_detect(words, "[aeiou]")`.

7.3.2 Extracting matches

`str_extract()`: first match (`NA` if none); `str_extract_all()`: all, as a list (matrix with `simplify = TRUE`). Colours in the 720 `sentences`:

```
colours <- c("red", "orange", "yellow", "green", "blue", "purple")
colour_match <- str_c(colours, collapse = "|")
has_colour <- str_subset(sentences, colour_match)
head(str_extract(has_colour, colour_match))
```

```
## [1] "blue" "blue" "red"  "red"  "red"  "blue"
```

(It matches "red" in "flickered"; word boundaries `\b` fix that [Supplement].)

7.3.3 Grouped matches

`str_match()`: full match and groups as matrix columns. Noun phrase, "a"/"the" + word:

```
noun <- "(a|the) ([^ ]+)"
sentences %>% str_subset(noun) %>% head(4) %>% str_match(noun)
```

```
##      [,1]         [,2]  [,3]
## [1,] "the smooth" "the" "smooth"
## [2,] "the sheet"  "the" "sheet"
## [3,] "the depth"  "the" "depth"
## [4,] "a chicken"  "a"   "chicken"
```

For tibbles, `tidyr::extract()` names the groups as columns:

```
tibble(sentence = sentences) %>%
  extract(sentence, c("article", "noun"), "(a|the) ([^ ]+)", remove = FALSE)
```

```
## # A tibble: 720 x 3
##   sentence                                  article noun
##   <chr>                                     <chr>   <chr>
## 1 The birch canoe slid on the smooth planks. the     smooth
## 2 Glue the sheet to the dark blue background. the     sheet
## 3 It's easy to tell the depth of a well.      the     depth
## 4 These days a chicken leg is a rare dish.    a       chicken
## 5 Rice is often served in round bowls.        <NA>    <NA>
## # i 715 more rows
```

7.3.4 Replacing and splitting

`str_replace()`: first match, `str_replace_all()`: all; a named vector does several, backreferences reorder:

```
x <- c("1 house", "2 cars", "3 people")
str_replace_all(x, c("1" = "one", "2" = "two", "3" = "three"))
```

```
## [1] "one house"   "two cars"    "three people"
```

```
sentences %>%
  str_replace("([^ ]+) ([^ ]+) ([^ ]+)", "\\1 \\3 \\2") %>%   # swap 2nd and 3rd words
  head(3)
```

```
## [1] "The canoe birch slid on the smooth planks."
## [2] "Glue sheet the to the dark blue background."
## [3] "It's to easy tell the depth of a well."
```

`str_split()` returns a list (piece counts differ); `boundary("word")` splits on words:

```
"a|b|c|d" %>% str_split("\\|")
```

```
## [[1]]
```

```
## [1] "a" "b" "c" "d"
```

```
x <- "This is a sentence.  This is another sentence."
str_split(x, boundary("word"))[[1]]
```

```
## [1] "This"     "is"       "a"        "sentence" "This"     "is"       "another"
## [8] "sentence"
```

`str_locate()`: start and end positions, for `str_sub()`.

7.4 Selected exercises

Motivation. A regex is read twice, by R's string parser and by the regex engine.

Exercise 7.1 (R4DS Exercise 14.3.1.1.1)

Explain why each of the strings `"\"`, `"\\"` and `"\\\"` fails to match a literal backslash.

```
x <- "a\\b"
writeLines(x)
```

```
## a\b
```

```
str_detect(x, "\\")
```

```
## Error in stri_detect_regex(string, pattern, negate = negate, opts_regex = opts(pattern)):
     Unrecognized backslash escape sequence in pattern. (U_REGEX_BAD_ESCAPE_SEQUENCE,
    context=`\`)
```

```
str_detect(x, "\\\\")
```

```
## [1] TRUE
```

Solution.

Step 1. `"\"`: the backslash escapes the closing quote; the string never ends (+ prompt).

Step 2. `"\\"`: a valid one-backslash string, but as a regex an escape with nothing to escape: error.

Step 3. `"\\\"`: a backslash, then an escaped closing quote: unterminated.

Step 4. The regex is `\\`; escaping both characters in a string gives `"\\\\"`, which matches. ■

Exercise 7.2 (R4DS Exercise 14.3.5.1.1)

Describe in words what these expressions match: `(.)\1\1`, `"(.)(.)\\2\\1"`, `(..)\1`, `"(.).\\1.\\1"`, `"(.)(.)(.).*\\3\\2\\1"`. Check the answers on suitable strings.

Solution. Expressions 1 and 3 are regexes; the others are R strings (doubled backslashes become single). As regexes:

Step 1. `(.)\1\1`: one character three times, "aaa".

Step 2. `(.)(.)\2\1`: two characters then the same reversed, "abba".

Step 3. `(..)\1`: a repeated pair, "abab".

Step 4. `(.).\1.\1`: a, any, a, any, a: "abaca".

Step 5. `(.)(.)(.).*\3\2\1`: three characters, anything (possibly nothing), the three reversed: "abcxcba", "abccba".

```
x <- c("aaa", "abba", "abab", "abaca", "abcxcba")
pats <- c(p1 = "(.)\\1\\1", p2 = "(.)(.)\\2\\1", p3 = "(..)\\1",
```

```
          p4 = "(.).\\1.\\1", p5 = "(.)(.)(.).*\\3\\2\\1")
sapply(pats, function(p) str_detect(x, p)) %>% `rownames<-`(x)
```

```
##           p1    p2    p3    p4    p5
## aaa     TRUE FALSE FALSE FALSE FALSE
## abba   FALSE  TRUE FALSE FALSE FALSE
## abab   FALSE FALSE  TRUE FALSE FALSE
## abaca  FALSE FALSE FALSE  TRUE FALSE
## abcxcba FALSE FALSE FALSE FALSE  TRUE
```

Diagonal: each string matches only its own pattern ("abba" fails `p5`, needing six characters; "aaa" fails `p3`, needing four). ■

Exercise 7.3 (one pattern or several; R4DS Exercise 14.4.1.1.1)

(a) Prove that for every string, `str_detect(s, "^x|x$")` equals `str_detect(s, "^x") | str_detect(s, "x$")`.
(b) Prove that `str_detect(s, "^[aeiou].*[^aeiou]$")` equals `str_detect(s, "^[aeiou]") & str_detect(s, "[^aeiou]$")` for every string s without newlines.
(c) Check both on `words`.

Solution [Supplement].

(a) s matches `A|B` iff it contains a match of `A` or of `B`; here "x at the start" or "x at the end": the same condition.

(b)

Step 1. If s matches the pattern, its first character is a vowel and its last is not: both tests `TRUE`.

Step 2. Conversely, a vowel first and a non-vowel last are different characters, so $n \geq 2$; `^[aeiou]` takes character 1, `.*` characters $2, \ldots, n-1$ (possibly none; no newlines), `[^aeiou]$` character n.

Step 3. If either test fails, the pattern fails by Step 1; the sides agree for every s.

(c)

```
a1 <- str_detect(words, "^x|x$")
a2 <- str_detect(words, "^x") | str_detect(words, "x$")
b1 <- str_detect(words, "^[aeiou].*[^aeiou]$")
b2 <- str_detect(words, "^[aeiou]") & str_detect(words, "[^aeiou]$")
c(identical(a1, a2), identical(b1, b2))
```

```
## [1] TRUE TRUE
```

```
words[a1]
```

```
## [1] "box" "sex" "six" "tax"
```

```
sum(b1)
```

```
## [1] 122
```

The combined calls are easier to read and modify. ■